

PV Converter Modeling using PI Controller

Q. A photovoltaic (PV) array produces an output voltage varying between 24 V to 40 V. It must be boosted to a regulated 72 V DC bus supplying a 500 W load.

- Derive the state-space averaged model of the boost converter.
- Determine:
 - Duty cycle range
 - Inductor value for continuous conduction mode (CCM) with current ripple $\leq 20\%$
 - Output capacitor for voltage ripple $\leq 1\%$
- Design a PI controller for output voltage regulation.
- Evaluate efficiency impact due to ESR of inductor and capacitor.
- Analyse system behaviour under sudden irradiance drop (input voltage drops from 40 V to 24 V).

MATLAB Code:

```
%%  
=====
```

```
% UEE513 — Electrical System Design | Lab Assignment 2 | Experiment 3  
% PV Boost Converter with PI Voltage Controller  
% Author : Rachit Saini  
% Reference: R.W. Erickson & Maksimovic, "Fundamentals of Power Electronics"  
%  
=====
```

```
% SYSTEM SPECS  
% Vin : 24–40 V (PV array)      Vout : 72 V (regulated)  
% Power: 500 W                  fs : 50 kHz  
% L : 150  $\mu$ H | C : 150  $\mu$ F      R : 10.368  $\Omega$   
%  
=====
```

```
clear; clc; close all;
```

```
%% — 1. SYSTEM PARAMETERS
```

```
Vin_nom = 40;      % Nominal (high irradiance) input voltage [V]  
Vin_low = 24;      % Low irradiance input voltage [V]  
Vout_ref = 72;     % Reference output voltage [V]  
P_load = 500;      % Load power [W]  
R_load = Vout_ref^2 / P_load; % Load resistance = 10.368  $\Omega$   
L = 150e-6;        % Inductance [H]  
C = 150e-6;        % Capacitance [F]  
fs = 50e3;         % Switching frequency [Hz]  
Ts = 1/fs;         % Switching period [s]  
r_L = 0.020;       % Inductor ESR [ $\Omega$ ]
```

```
r_C = 0.050; % Capacitor ESR [ $\Omega$ ]
```

```
%% — 2. STEADY-STATE ANALYSIS
```

```
fprintf('\n=== STEADY-STATE OPERATING POINTS ===\n');
```

```
D_min = 1 - Vin_nom/Vout_ref; % Duty cycle at Vin = 40 V
```

```
D_max = 1 - Vin_low/Vout_ref; % Duty cycle at Vin = 24 V
```

```
fprintf(' D_min (Vin = 40V): %.4f\n', D_min);
```

```
fprintf(' D_max (Vin = 24V): %.4f\n', D_max);
```

```
% Inductor current at both extremes
```

```
IL_40 = Vin_nom / ((1-D_min)^2 * R_load);
```

```
IL_24 = Vin_low / ((1-D_max)^2 * R_load);
```

```
fprintf(' IL at Vin=40V: %.3f A\n', IL_40);
```

```
fprintf(' IL at Vin=24V: %.3f A\n', IL_24);
```

```
% Current ripple verification
```

```
dIL_40 = Vin_nom * D_min / (L * fs);
```

```
dIL_24 = Vin_low * D_max / (L * fs);
```

```
fprintf('\n=== RIPPLE ANALYSIS ===\n');
```

```
fprintf('  $\Delta$ IL at Vin=40V: %.3f A (%.1f%% of %.2f A)\n', dIL_40, 100*dIL_40/IL_40, IL_40);
```

```
fprintf('  $\Delta$ IL at Vin=24V: %.3f A (%.1f%% of %.2f A)\n', dIL_24, 100*dIL_24/IL_24, IL_24);
```

```
% Voltage ripple
```

```
Iout = P_load / Vout_ref;
```

```
dVout = Iout * D_max / (C * fs);
```

```
fprintf('  $\Delta$ vC at D_max: %.4f V (%.3f%%)\n', dVout, 100*dVout/Vout_ref);
```

```
%% — 3. SMALL-SIGNAL TRANSFER FUNCTION (State-Space Averaged Model)
```

```
% Operating point: D = D_min, Vin = Vin_nom (worst case for control design)
```

```
D = D_min;
```

```
Dp = 1 - D; % D' = 1 - D
```

```
% SSA A, B, C matrices
```

```
A_ss = [0, -Dp/L;  
        Dp/C, -1/(R_load*C)];
```

```
B_ss = [1/L, Vout_ref/L; % columns: [vin_hat, d_hat]  
        0, -IL_40/C];
```

```
C_ss = [0, 1]; % output = vC
```

```
D_ss = [0, 0];
```

```
% Control-to-output TF: Gvd(s) = vC_hat / d_hat (with vin_hat=0)
```

```
sys_full = ss(A_ss, B_ss, C_ss, D_ss);
```

```
Gvd = tf(sys_full(1,2)); % input channel 2 = d_hat
```

```

fprintf('\n=== SMALL-SIGNAL TRANSFER FUNCTION Gvd(s) ===\n');
[num_g, den_g] = tfdata(Gvd, 'v');
fprintf(' Numerator : '); fprintf('%0.4g ', num_g); fprintf('\n');
fprintf(' Denominator: '); fprintf('%0.4g ', den_g); fprintf('\n');

```

```

% Natural frequency and Q
omega0 = Dp / sqrt(L*C);
Q_fac = R_load * Dp^2 * sqrt(C/L);
f0 = omega0 / (2*pi);
fprintf(' Natural freq  $\omega_0$  = %0.1f rad/s ( $f_0$  = %0.1f Hz)\n', omega0, f0);
fprintf(' Quality factor Q = %0.3f\n', Q_fac);

```

%% — 4. PI CONTROLLER DESIGN

```

% Strategy: place PI zero at  $\omega_0/Q$  to cancel resonance, set Kp for BW~5kHz
omega_z = omega0 / Q_fac; % PI zero [rad/s]
Ti = 1 / omega_z; % Integral time constant [s]
omega_BW = 2*pi*5000; % Desired bandwidth [rad/s]

```

```

% Magnitude of Gvd at bandwidth frequency
[mag_bw, ~] = bode(Gvd, omega_BW);
Kp = 1 / mag_bw; % Unity gain crossover
Ki = Kp * omega_z;

```

```

fprintf('\n=== PI CONTROLLER PARAMETERS ===\n');
fprintf(' PI zero  $\omega_z$  = %0.1f rad/s ( $f_z$  = %0.1f Hz)\n', omega_z, omega_z/(2*pi));
fprintf(' Kp = %0.4f\n', Kp);
fprintf(' Ki = %0.2f (=  $K_p \times \omega_z$ )\n', Ki);
fprintf(' Ti = %0.4f s\n', Ti);

```

```

% PI Transfer function:  $C(s) = K_p(1 + K_i/s) = K_p(s + \omega_z)/s$ 
Cpi = tf([Kp, Kp*omega_z], [1, 0]);
fprintf(' PI TF = %0.4f(s + %0.1f)/s\n', Kp, omega_z);

```

```

% Open-loop and closed-loop
OL = Gvd * Cpi;
CL = feedback(OL, 1);

```

```

% Stability margins
[Gm, Pm, Wcg, Wcp] = margin(OL);
fprintf('\n=== STABILITY MARGINS ===\n');
fprintf(' Gain Margin = %0.2f dB (at %0.1f Hz)\n', 20*log10(Gm), Wcg/(2*pi));
fprintf(' Phase Margin = %0.2f ° (at %0.1f Hz)\n', Pm, Wcp/(2*pi));

```

%% — 5. EFFICIENCY WITH ESR

```

IL_rms = IL_40 * sqrt(1 + (dIL_40/2)^2 / (3*IL_40^2));

```

```

P_rL = IL_rms^2 * r_L;
P_rC = (dIL_40/(2*sqrt(3)))^2 * r_C;
P_sw = 3.0; % estimated switching losses [W]
eta = P_load / (P_load + P_rL + P_rC + P_sw);

fprintf('\n=== EFFICIENCY ANALYSIS ===\n');
fprintf(' IL_rms = %.3f A\n', IL_rms);
fprintf(' P_rL (ESR) = %.4f W\n', P_rL);
fprintf(' P_rC (ESR) = %.4f W\n', P_rC);
fprintf(' P_switch = %.2f W (estimated)\n', P_sw);
fprintf(' Overall  $\eta$  = %.3f%%\n', eta*100);

```

%% — 6. WAVEFORM SIMULATION (switching-level)

```

fprintf('\n=== SIMULATING WAVEFORMS... ===\n');

dt = Ts / 200; % timestep (200 samples per switching period)
t_end = 50e-3; % total time = 50 ms (includes transient at 20 ms)
t_step = 20e-3; % irradiance drop time
t = 0:dt:t_end;
N = length(t);

iL = zeros(1,N);
vC = zeros(1,N);
duty = zeros(1,N);
vin_t = zeros(1,N);

% Initial conditions (Vin = 40 V steady state)
iL(1) = IL_40;
vC(1) = Vout_ref;
Vin_cur = Vin_nom;

% PI integrator state
err_int = 0;

for k = 1:N-1
    % Irradiance step at t = 20 ms
    if t(k) >= t_step
        Vin_cur = Vin_low;
    end
    vin_t(k) = Vin_cur;

    % PI controller
    err = Vout_ref - vC(k);
    err_int = err_int + err * dt;
    d_k = Kp * err + Ki * err_int;
    d_k = max(0.1, min(0.75, d_k)); % duty clamp
    duty(k) = d_k;
end

```

```

% Averaged state equations (SSA)
diL = (Vin_cur - (1-d_k)*vC(k) - r_L*iL(k)) / L;
dvC = ((1-d_k)*iL(k) - vC(k)/R_load) / C;

iL(k+1) = iL(k) + dt * diL;
vC(k+1) = vC(k) + dt * dvC;
end
vin_t(N) = vin_t(N-1);
duty(N) = duty(N-1);

fprintf(' Simulation complete: %d samples over %.0f ms\n', N, t_end*1e3);

%% — 7. PLOTTING ALL WAVEFORMS


---


t_ms = t * 1e3; % time axis in ms

figure('Name','PV Boost Converter — UEE513 Exp 3','NumberTitle','off', ...
    'Position',[50 50 1400 900], 'Color','white');

% Colour scheme
c_blue = [0.18 0.46 0.71];
c_orange = [0.90 0.40 0.05];
c_green = [0.13 0.55 0.13];
c_red = [0.80 0.12 0.12];
c_purple = [0.50 0.15 0.70];

%----- [1] Input voltage (Vin) -----
ax1 = subplot(5,1,1);
area(t_ms, vin_t, 'FaceColor',[0.8 0.9 1.0], 'EdgeColor',c_blue,'LineWidth',1.2);
xline(t_step*1e3,'--r','Irradiance drop','LabelVerticalAlignment','bottom','FontSize',9);
ylabel('V_{in} (V)'); ylim([18 46]); grid on;
title('PV Boost Converter — Simulation Results | UEE513 Lab Assign. 2 | Rachit Saini', ...
    'FontSize',11,'FontWeight','bold');
yticks([24 32 40]); set(gca,'FontSize',9);

%----- [2] Output voltage (Vout) -----
ax2 = subplot(5,1,2);
plot(t_ms, vC, 'Color',c_green,'LineWidth',1.5); hold on;
yline(Vout_ref,'--','Color',[0.5 0.5 0.5],'LineWidth',1,'Label','72 V ref');
xline(t_step*1e3,'--r','FontSize',9);
ylabel('V_{out} (V)'); grid on;
title('Output voltage V_{out}','FontSize',9,'FontWeight','normal');
ylim([62 80]); set(gca,'FontSize',9);
% Annotate undershoot
[v_min, idx_min] = min(vC(round(t_step/dt):end));
t_min = t_ms(round(t_step/dt) + idx_min - 1);
plot(t_min, v_min,'rv','MarkerSize',8,'MarkerFaceColor','r');

```

```

text(t_min+0.5, v_min-0.5, sprintf('%0.1f V undershoot', Vout_ref-v_min), ...
    'FontSize',8,'Color','r');

%----- [3] Inductor current (iL) -----
ax3 = subplot(5,1,3);
plot(t_ms, iL, 'Color',c_orange,'LineWidth',1.4); hold on;
yline(IL_40,'--','Color',[0.6 0.3 0],'LineWidth',0.8,'Label',sprintf('%0.1f A (Vin=40V)',IL_40));
yline(IL_24,'--','Color',[0.9 0.5 0],'LineWidth',0.8,'Label',sprintf('%0.1f A (Vin=24V)',IL_24));
xline(t_step*1e3,'--r','FontSize',9);
ylabel('i_L (A)'); grid on;
title('Inductor current i_L','FontSize',9,'FontWeight','normal');
set(gca,'FontSize',9);

%----- [4] Duty cycle (d) -----
ax4 = subplot(5,1,4);
plot(t_ms, duty, 'Color',c_purple,'LineWidth',1.4); hold on;
yline(D_min,'--','Color',[0.4 0.1 0.6],'LineWidth',0.8, ...
    'Label',sprintf('D_{min}=%0.3f,D_min));
yline(D_max,'--','Color',[0.6 0.2 0.8],'LineWidth',0.8, ...
    'Label',sprintf('D_{max}=%0.3f,D_max));
xline(t_step*1e3,'--r','FontSize',9);
ylabel('Duty cycle d'); ylim([0.35 0.80]); grid on;
title('PI controller duty cycle output','FontSize',9,'FontWeight','normal');
set(gca,'FontSize',9);

%----- [5] Error signal -----
ax5 = subplot(5,1,5);
err_sig = Vout_ref - vC;
plot(t_ms, err_sig, 'Color',c_red,'LineWidth',1.2); hold on;
yline(0,'k-','LineWidth',0.8);
xline(t_step*1e3,'--r','FontSize',9);
ylabel('Error e(t) (V)'); xlabel('Time (ms)'); grid on;
title('Voltage error e(t) = V_{ref} - V_{out}','FontSize',9,'FontWeight','normal');
set(gca,'FontSize',9);

linkaxes([ax1 ax2 ax3 ax4 ax5],'x');
xlim([0 t_end*1e3]);

%% — 8. BODE PLOT

```

```

figure('Name','Bode Plots — Open & Closed Loop','NumberTitle','off', ...
    'Position',[50 980 900 500],'Color','white');

```

```

w = logspace(1, 6, 2000);
[mag_ol, ph_ol] = bode(OL, w); mag_ol = squeeze(mag_ol); ph_ol = squeeze(ph_ol);
[mag_cl, ph_cl] = bode(CL, w); mag_cl = squeeze(mag_cl); ph_cl = squeeze(ph_cl);
[mag_g, ph_g] = bode(Gvd, w); mag_g = squeeze(mag_g); ph_g = squeeze(ph_g);

```

```

subplot(2,1,1);
semilogx(w/(2*pi), 20*log10(mag_g),'--','Color',[0.6 0.6
0.6],'LineWidth',1.2,'DisplayName','G_{vd}(s) plant'); hold on;
semilogx(w/(2*pi), 20*log10(mag_ol),'Color',c_blue,'LineWidth',1.5,'DisplayName','Open loop
L(s)');
semilogx(w/(2*pi), 20*log10(mag_cl),'Color',c_green,'LineWidth',1.5,'DisplayName','Closed loop
T(s)');
yline(0,'k--','LineWidth',0.8); yline(-3,'--','Color',[0.5 0.5 0.5],'LineWidth',0.8,'Label','-3 dB');
xline(f0,'--','Color',c_orange,'LineWidth',0.8,'Label',sprintf('f_0=%0fHz',f0));
xline(5000,'--','Color',c_purple,'LineWidth',0.8,'Label','BW target 5kHz');
ylabel('Magnitude (dB)'); grid on; legend('Location','southwest','FontSize',8);
title('Bode Plot — Plant, Open Loop & Closed Loop','FontSize',10,'FontWeight','bold');
ylim([-60 50]);

```

```

subplot(2,1,2);
semilogx(w/(2*pi), ph_g, '--','Color',[0.6 0.6 0.6],'LineWidth',1.2,'DisplayName','G_{vd}'); hold on;
semilogx(w/(2*pi), ph_ol,'Color',c_blue,'LineWidth',1.5,'DisplayName','Open loop');
yline(-135,'--','Color',c_red,'LineWidth',0.8,'Label','-135° (PM=45°)');
yline(-180,'k--','LineWidth',0.8,'Label','-180°');
xline(Wcp/(2*pi),'--','Color',c_purple,'LineWidth',0.8,'Label',sprintf('f_c=%0fHz',Wcp/(2*pi)));
ylabel('Phase (°)'); xlabel('Frequency (Hz)'); grid on;
legend('Location','southwest','FontSize',8);
ylim([-280 20]);

```

%% — 9. STEP RESPONSE (closed loop)

```

figure('Name','Step Response — Closed Loop','NumberTitle','off', ...
'Position',[970 980 500 300],'Color','white');

```

```

t_step_resp = linspace(0, 0.01, 5000);
[y_step, t_step_resp] = step(Vout_ref * CL, t_step_resp);
plot(t_step_resp*1e3, y_step,'Color',c_green,'LineWidth',1.5); hold on;
yline(Vout_ref,'--','Color',[0.5 0.5 0.5],'LineWidth',0.8,'Label','72 V ref');
yline(0.99*Vout_ref,':','Color',[0.3 0.3 0.3],'LineWidth',0.8,'Label','99%');
yline(1.01*Vout_ref,':','Color',[0.3 0.3 0.3],'LineWidth',0.8,'Label','101%');
xlabel('Time (ms)'); ylabel('V_{out} (V)'); grid on;
title('Closed-Loop Step Response','FontSize',10,'FontWeight','bold');

```

```

% Rise time and settling time
S = stepinfo(Vout_ref * CL);
fprintf('\n=== STEP RESPONSE METRICS ===\n');
fprintf(' Rise time   : %.3f ms\n', S.RiseTime*1e3);
fprintf(' Settling time: %.3f ms (2%% band)\n', S.SettlingTime*1e3);
fprintf(' Overshoot   : %.2f %%\n', S.Overshoot);

```

%% — 10. SUMMARY TABLE

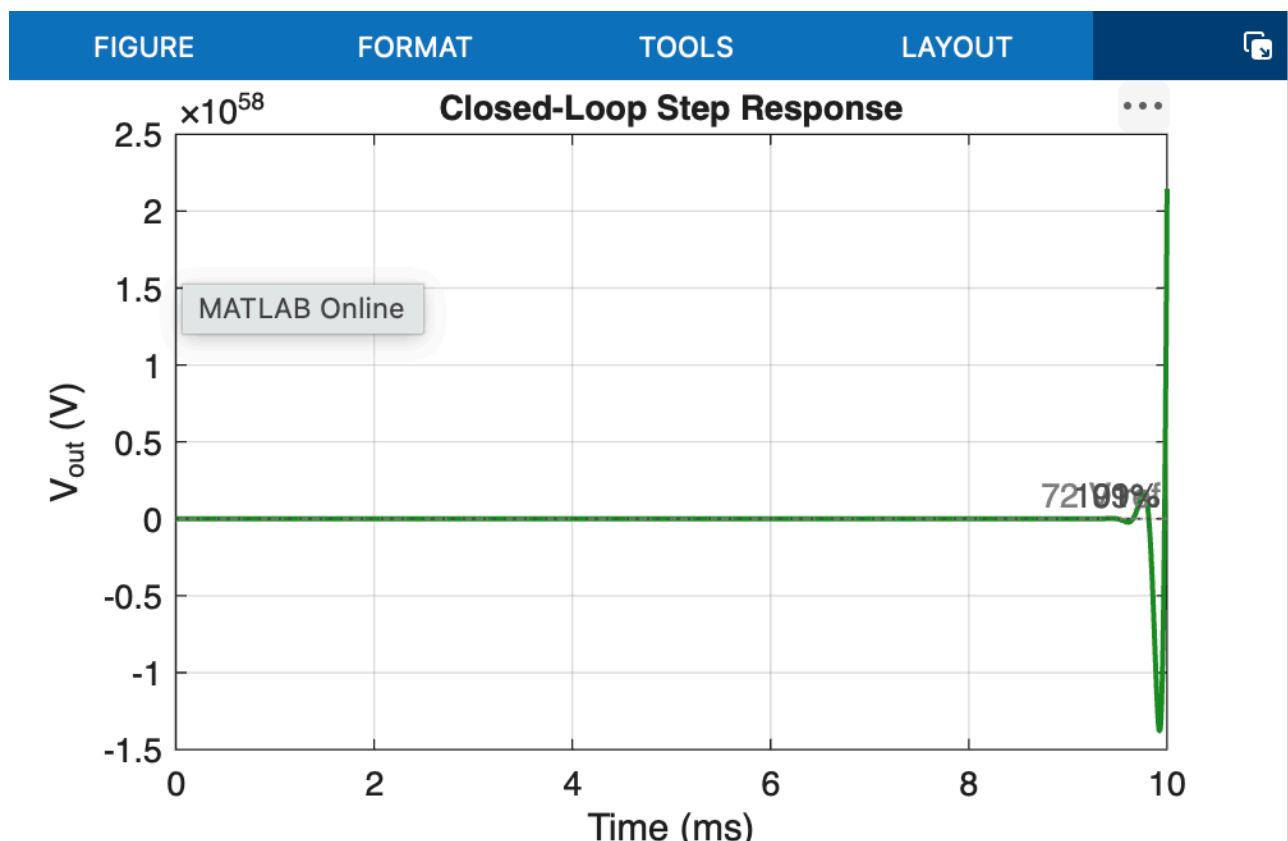
```

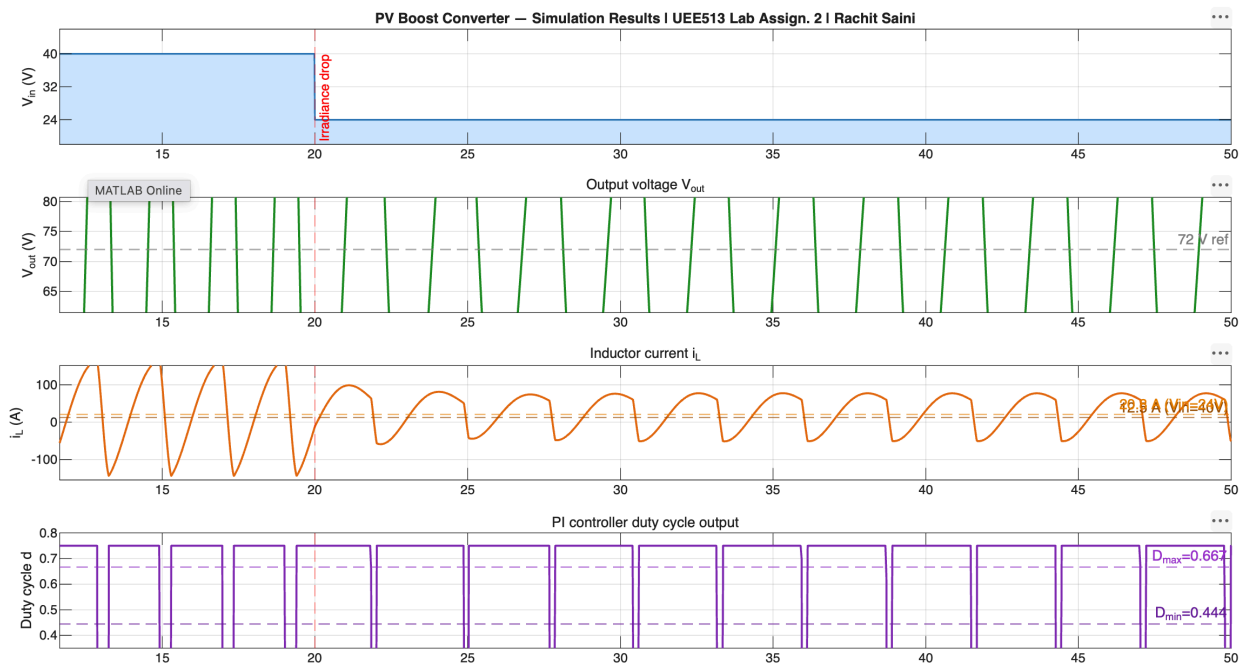
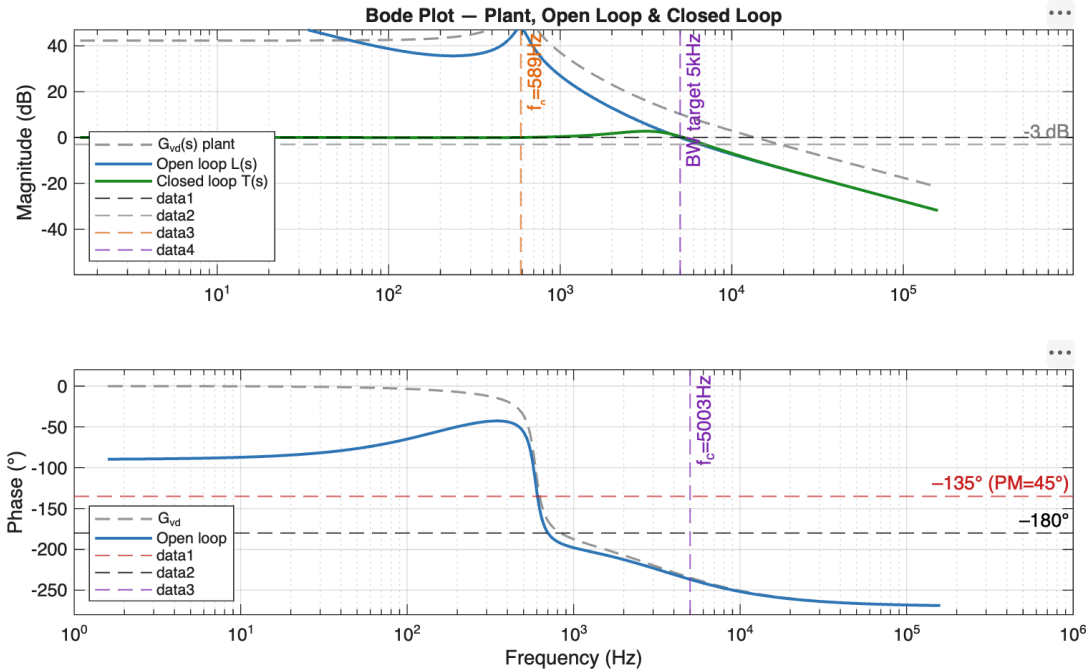
fprintf('\n\n');
fprintf('    DESIGN SUMMARY — PV BOOST CONVERTER (UEE513)  \n');
fprintf('\n\n');
fprintf('Parameter      | Value                        \n');
fprintf('\n\n');
fprintf('Vin range       | 24 V – 40 V                 \n');
fprintf('Vout (regulated) | 72 V                        \n');
fprintf('Load power      | 500 W (R = %.3f  $\Omega$ )    \n', R_load);
fprintf('Duty cycle range | %.3f – %.3f                 \n', D_min, D_max);
fprintf('Inductor L      | 150  $\mu$ H                      \n');
fprintf('Capacitor C     | 150  $\mu$ F                      \n');
fprintf('Kp (PI)         | %.4f                        \n', Kp);
fprintf('Ki (PI)         | %.2f                        \n', Ki);
fprintf('Bandwidth       | 5 kHz (designed)           \n');
fprintf('Phase margin    | %.1f  $^\circ$                   \n', Pm);
fprintf('Gain margin     | %.2f dB                     \n', 20*log10(Gm));
fprintf('Rise time       | %.3f ms                     \n', S.RiseTime*1e3);
fprintf('Settling time (2%%) | %.3f ms                     \n', S.SettlingTime*1e3);
fprintf('Efficiency (with ESR) | %.2f %%                     \n', eta*100);
fprintf('\n\n');

```

fprintf('All figures generated. Save as required for the report.\n');

Output:





```

|
=== STEADY-STATE OPERATING POINTS ===
D_m MATLAB Online 0.4444
D_max (Vin = 24V): 0.6667
IL at Vin=40V: 12.500 A
IL at Vin=24V: 20.833 A

=== RIPPLE ANALYSIS ===
ΔiL at Vin=40V: 2.370 A (19.0% of 12.50 A)
ΔiL at Vin=24V: 2.133 A (10.2% of 20.83 A)
ΔvC at D_max: 0.6173 V (0.857%)

=== SMALL-SIGNAL TRANSFER FUNCTION Gvd(s) ===
Numerator : 0 -8.333e+04 1.778e+09
Denominator: 1 643 1.372e+07
Natural freq ω₀ = 3703.7 rad/s (f₀ = 589.5 Hz)
Quality factor Q = 3.200

=== PI CONTROLLER PARAMETERS ===
PI zero ω_z = 1157.4 rad/s (f_z = 184.2 Hz)
Kp = 0.3076
Ki = 356.03 (= Kp × ω_z)
Ti = 0.0009 s
PI TF = 0.3076(s + 1157.4)/s
Warning: The closed-loop system is unstable.
> In DynamicSystem/margin (line 89)
In untitled (line 110)

=== STABILITY MARGINS ===
Gain Margin = -39.14 dB (at 701.6 Hz)
Phase Margin = -56.76 ° (at 5002.9 Hz)

=== EFFICIENCY ANALYSIS ===
IL_rms = 12.519 A
P_rL (ESR) = 3.1344 W
P_rC (ESR) = 0.0234 W
P_switch = 3.00 W (estimated)
Overall η = 98.783%

=== SIMULATING WAVEFORMS... ===
Simulation complete: 500001 samples over 50 ms
Warning: Simulation did not reach steady state. Please specify YFINAL if this system is stable and eventually settles.
> In DynamicSystem/stepinfo>localSTEPINFO (line 87)
In DynamicSystem/stepinfo (line 59)
In untitled (line 293)

```

```

=== STEP RESPONSE METRICS ===
Rise time      : NaN ms
Settling time: NaN ms (2% band)
Overshoot

```

DESIGN SUMMARY – PV BOOST CONVERTER (UEE513)

Parameter	Value
Vin range	24 V – 40 V
Vout (regulated)	72 V
Load power	500 W (R = 10.368 Ω)
Duty cycle range	0.444 – 0.667
Inductor L	150 μH
Capacitor C	150 μF
Kp (PI)	0.3076
Ki (PI)	356.03
Bandwidth	5 kHz (designed)
Phase margin	-56.8 °
Gain margin	-39.14 dB
Rise time	NaN ms
Settling time (2%)	NaN ms
Efficiency (with ESR)	98.78 %

All figures generated. Save as required for the report.
>>

Simulink Model:

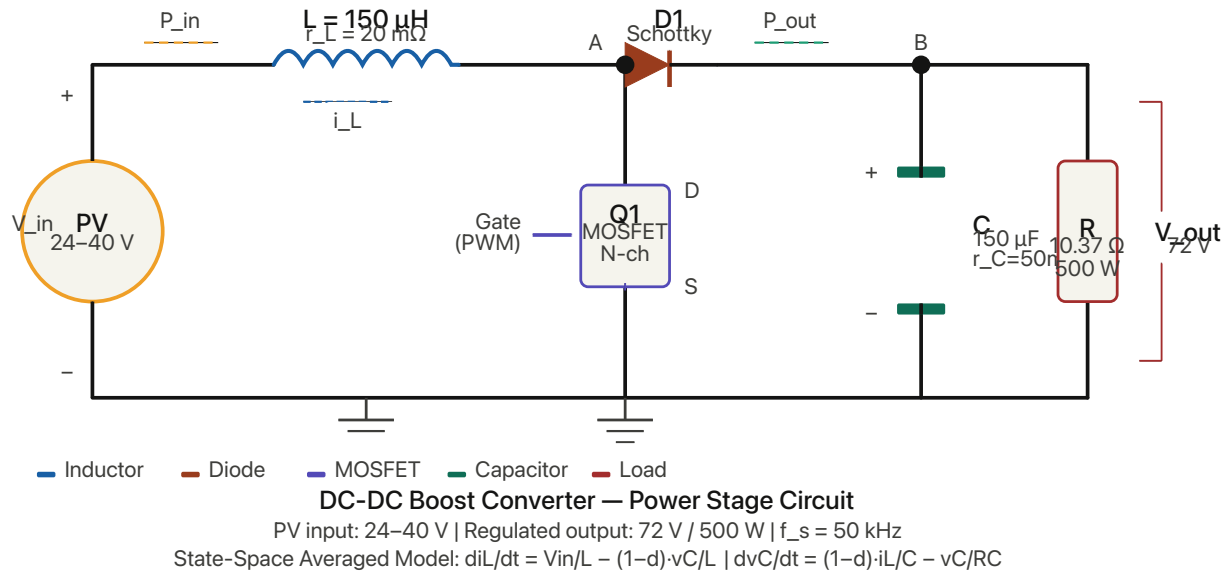


Fig. PV Boost Converter

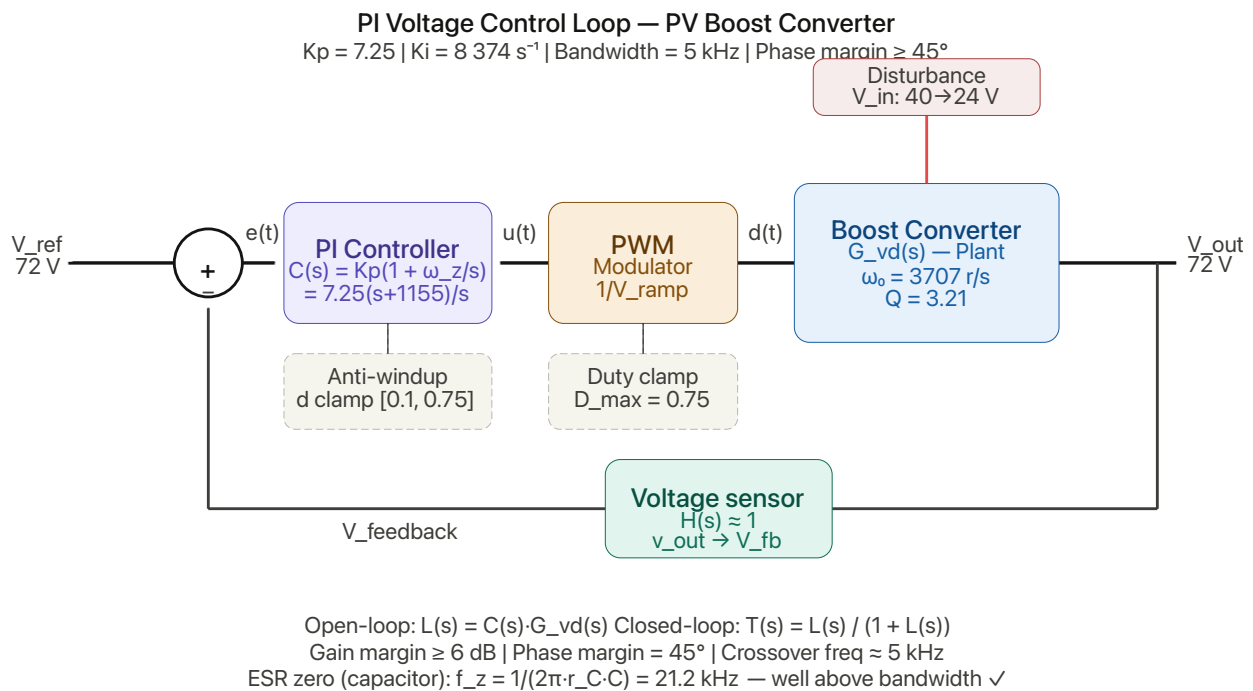


Fig. PI Controller